Làm rõ giao thức tấn công: IoT-23, vertical scan và máy chủ MQTT

Đối chiếu nguồn công bố, PCAP và code của nhóm • 09/11/2026

1. Kết luận cho đúng mục tiêu của nhóm

Mục tiêu được người dùng xác nhận là máy chủ MQTT phía sau gateway, không phải bo FPGA tự chạy MQTT broker.

Thầy có lý khi yêu cầu nhóm làm rõ giao thức và bằng chứng gây gián đoạn. Hai mẫu tấn công đang dùng là TCP SYN flood và TCP SYN vertical scan. Chúng chưa phải các cuộc tấn công bằng bản tin MQTT ở tầng ứng dụng; dữ liệu hiện có cũng không chứng minh máy chủ MQTT của nhóm đã bị sập.

Tuy nhiên, không thể coi "MQTT không thể bị sập hoặc DoS" là một quy luật chung. MQTT là giao thức ứng dụng; broker, hệ điều hành và mạng có tài nguyên hữu hạn. Đã có lỗi MQTT gây crash được nhà phát triển broker công bố và vá. Khả năng ảnh hưởng phụ thuộc kiểu tấn công, tải, phiên bản, cấu hình và cách triển khai; không thể suy ra từ tên giao thức.

Tài liệu này không chạy thử tấn công, không tạo lưu lượng mới và không thay model. Kết luận về PCAP dựa trên phép đọc ngoại tuyến.

2. Đã đọc những gì, ở mức nào?

Phạm vi	Công việc đã hoàn thành
Toàn bộ danh mục IoT-23	Rà tài liệu chính thức, chỉ mục 23 kịch bản, README của 20 kịch bản malware và thông tin ba nhóm thiết bị bình thường; lưu bảng danh mục 23 dòng
Nhãn của 20 kịch bản malware	Đọc và trích đầy đủ các bảng phân bố nhãn do nhà công bố cung cấp
Toàn bộ PCAP capture 34 có sẵn	Đọc 233.865 frame, kích thước 126.317.980 byte; thống kê TCP/UDP và cổng
Đoạn SYN mà model sử dụng	Đọc đủ 40.568 gói trong file đã lọc và xác nhận cờ TCP, đích, cổng, kích thước/payload
Mẫu vertical scan	Đọc đủ 2.011 frame và đối chiếu file nguồn Chris Sanders
Lab MQTT đã gửi	Đọc mã tạo lưu lượng, MQTT client, cấu hình broker và cách đăng ký capture

Không tuyên bố đã đọc từng packet của toàn bộ gói IoT-23 đầy đủ khoảng 21 GB nén. Phần toàn bộ dataset trong báo cáo là mô tả, chỉ mục và nhãn công bố; phần kiểm packet trực tiếp có phạm vi cụ thể như bảng trên. Không kết luận "toàn IoT-23 tuyệt đối không có MQTT". Một README không nhắc MQTT hoặc một cổng không phải 1883 đều không đủ để chứng minh điều đó.

Somfy gồm sáu thư mục con; không có README tại các thư mục con đã duyệt, nên phần thiết bị này dùng mô tả nhà công bố và chỉ mục. Các kết quả không được mô tả thành “đọc README của đủ 23 capture”.

3. Người ta tạo dữ liệu IoT-23 như thế nào?

Theo Stratosphere Laboratory/CTU và bản ghi Zenodo, IoT-23 có 20 kịch bản malware và ba kịch bản bình thường, thu trong giai đoạn 2018–2019. Mỗi kịch bản độc hại thực thi một mẫu malware trên Raspberry Pi rồi ghi lại lưu lượng. Mạng được kiểm soát nhưng có kết nối Internet không bị giới hạn như mô tả nguồn. Dữ liệu bình thường được thu từ thiết bị thật: Philips HUE, Amazon Echo và khóa Somfy.

Phần tổng quan giao thức ứng dụng của nhà công bố liệt kê HTTP, DNS, DHCP, Telnet, SSL, IRC và các luồng không được Zeek nhận diện. Phần không nhận diện khiến ta không thể kết luận mọi luồng còn lại đều không phải MQTT.

Đây là ghi nhận hành vi của thiết bị nhiễm và thiết bị lành, không phải một thiết kế thí nghiệm chuyên biệt để làm sập MQTT broker. Không được diễn giải rằng toàn bộ tập là “tấn công MQTT” chỉ vì tên có IoT.

PCAP được phân tích bằng Zeek và bằng người phân tích. Nhãn được thêm vào conn.log theo quy tắc của nhóm công bố. Các nhãn gồm Benign, C&C, DDoS, Attack, FileDownload, HeartBeat, PartOfAHorizontalPortScan và một số nhãn họ malware/kết hợp.

DDoS là nhãn hành vi, không tự chỉ ra TCP SYN, UDP hay giao thức ứng dụng. PartOfAHorizontalPortScan là quét ngang, không được đổi thẳng thành vertical scan. Cần kiểm gói để xác định giao thức và kiểu tấn công của đoạn thực sự dùng.

README capture 34 ghi probable malware Mirai, thiết bị nhiễm 192.168.1.195 và origin device RPi02. Capture gốc có 167.905 frame IPv4/TCP và 60.564 frame IPv4/UDP; không ghi gói TCP nhắm cổng 1883/8883 trong lần đếm này. Điều đó mô tả riêng capture 34, không phải toàn bộ IoT-23 và không loại trừ MQTT dùng cổng khác.

4. Bằng chứng của hai mẫu tấn công đang dùng

4.1. Đoạn SYN capture 34

Thuộc tính	Kết quả đọc file đã lọc
Nguồn	192.168.1.195
Đích	71.61.66.148
Giao thức	IPv4/TCP
Cổng đích	80
Số gói	40.568
Cờ TCP	Tất cả 0x02: SYN bật, ACK tắt
TCP application payload	0 byte ở mọi gói
Khoảng thời gian	Khoảng 22,03 giây

Vì không có TCP payload, đoạn này không có bản tin MQTT CONNECT/PUBLISH/SUBSCRIBE. Dù cổng đích là 80, cũng không gọi đây là một cuộc tấn công bằng HTTP: nó chưa gửi yêu cầu HTTP. Đây là hành vi SYN flood ở tầng TCP trong đoạn được dự án chọn.

PCAP/nhãn này không kèm số đo CPU, bộ nhớ, tình trạng tiến trình hoặc khả năng phục vụ của máy chủ đích. Vì vậy không dùng nó để khẳng định victim đã sập, càng không khẳng định MQTT broker của nhóm sẽ sập khi gặp cùng một mẫu.

4.2. Mẫu vertical scan của Chris Sanders

Thuộc tính	Kết quả đọc PCAP
Scanner	172.16.0.8
Đích	64.13.134.52
Giao thức	IPv4/TCP
Frame toàn file	2.011
SYN từ scanner tới đích	1.994
Cổng đích phân biệt của scanner	1.000
Phản hồi	12 SYN-ACK và 5 RST-ACK
TCP application payload	0 byte ở toàn file
Gói nhắm 1883/8883	0

Đếm cổng trên chiều scanner tới đích mới được 1.000; nếu cộng cả chiều trả lời sẽ lẫn cổng nguồn của scanner vào thống kê.

Hành vi phù hợp TCP SYN scan: một nguồn thăm nhiều cổng trên một host để tìm dịch vụ. Quét cổng thường là thăm dò, không mặc định có mục đích hoặc kết quả làm sập dịch vụ. Các SYN-ACK/RST-ACK chỉ thể hiện phản ứng ở TCP; không có phiên MQTT được trao đổi trong mẫu này.

README công khai của Chris Sanders xác nhận nguồn bộ packet nhưng không mô tả lệnh/cấu hình tạo riêng mẫu synscan. Vì vậy báo cáo không khẳng định tác giả dùng một công cụ, lệnh hay tốc độ cấu hình cụ thể chỉ từ suy đoán.

5. Lab và kiến trúc hiện tại thực sự làm gì?

Trong lab/lab_traffic.py, nhánh syn tạo IPv4/TCP với flags S, nhắm cổng 1883. Nhánh scan thay đổi cổng đích. Mã đó không thực hiện phiên MQTT và không gửi bản tin CONNECT/PUBLISH. Dùng cổng 1883 không tự biến SYN thành một gói ứng dụng MQTT.

Mở một PCAP bằng Wireshark hoặc chạy HOC_AI.py để tính từ đặc trưng là thao tác ngoại tuyến; bản thân chúng không gửi gói tới broker. Thí nghiệm dịch vụ cần lưu lượng thật trong lab, đúng IP/cổng máy chủ và đi qua gateway. Không suy ra broker bị ảnh hưởng chỉ từ một file mẫu hoặc kết quả class của AI.

Ngược lại, lab/sensor_client.py dùng paho-mqtt, kết nối broker và publish QoS 1 lên lab/temperature. Đây là luồng MQTT hợp lệ làm nền và giúp quan sát khả năng phục vụ.

Code lab đặt giới hạn tốc độ yêu cầu 500 gói/giây và giữ lại các bảo vệ TCP của hệ điều hành. Tốc độ thực cần đọc từ PCAP/source.log. Đây không phải bằng chứng tải đó chắc chắn làm sập broker. run_capture.sh còn yêu cầu tiến trình broker/observer/capture sống tới cuối để đăng ký dữ liệu; capture được ghi nhận không đồng nghĩa có sự cố dịch vụ.

README phần 1 mô tả Tang Nano 9K là bridge/firewall Ethernet trong suốt: quan sát header, đếm đặc trưng, chạy AI và áp policy. Bo không phải MQTT broker. Mục tiêu hợp lý là bảo vệ lưu lượng/dịch vụ phía sau nó.

Sáu đặc trưng hiện tại là tải gói, độ dài IP, tỷ lệ SYN và số nguồn/đích/cổng phân biệt. Chúng không đọc topic, CONNECT, PUBLISH, SUBSCRIBE hoặc QoS MQTT. Model đã học SYN/scan không tự trở thành bộ phát hiện mọi tấn công ứng dụng MQTT.

6. Thầy đúng ở đâu? Cần nói chính xác điều gì?

Nhận định	Đánh giá theo bằng chứng
“Mẫu nhóm dùng chưa phải tấn công bằng MQTT”	Đúng với hai mẫu attack và code tạo SYN/scan đã kiểm: đó là TCP SYN/scan, không chứa bản tin MQTT
“Nhóm chưa chứng minh broker MQTT bị sập”	Đúng: PCAP và kết quả AI hiện có chưa chứng minh sự cố broker
“Port scan phải làm broker sập mới là scan”	Không: scan là thăm dò dịch vụ; không cần gây sập để được nhận diện
“SYN flood không thể tác động dịch vụ MQTT”	Không đúng nếu khẳng định tuyệt đối: áp lực TCP/tài nguyên có thể làm khách mới không kết nối được; mức tác động phải đo
“MQTT luôn miễn nhiễm với DoS hoặc crash”	Không đúng như khẳng định chung; có tài nguyên hữu hạn và đã có lỗi broker MQTT gây crash được công bố

SYN flood thường gây áp lực vào quá trình thiết lập TCP mới. Một phiên MQTT đã kết nối có thể vẫn publish bình thường trong khi client mới khó kết nối. Vì vậy nhìn một client còn hoạt động không đủ chứng minh mọi thứ không bị ảnh hưởng; nhìn SYN nhiều cũng không đủ chứng minh broker sập.

Không phục vụ được client là DoS dù tiến trình vẫn sống. Crash là tiến trình bị dừng/lỗi. Chậm hơn, mất message, không mở được kết nối mới và crash là những kết quả khác nhau, cần ghi riêng.

Nguồn kiểm chứng cho nhận định MQTT không miễn nhiễm: ChangeLog chính thức Mosquitto ghi CVE-2021-28166 trong mục 2.0.10. Các bản broker 2.0.0–2.0.9 từng có lỗi xử lý MQTT v5 từ client đã xác thực có thể dẫn tới segfault; lỗi được vá ở 2.0.10. Đây là bằng chứng lịch sử, không phải kết luận broker hiện tại của nhóm còn lỗi này và không phải đề nghị dùng broker lỗi để thử.

7. Nhóm nên làm gì tiếp theo?

Nếu giữ đề tài SYN flood/scan, tên mô hình nên là “Phát hiện và chặn TCP SYN flood, TCP vertical scan để bảo vệ hệ thống IoT sử dụng MQTT”. MQTT là dịch vụ được bảo vệ; TCP SYN/scan là cơ chế tấn công. Lab một nguồn cần mô tả là một nguồn phát SYN/scan, không tự gọi là tấn công phân tán chỉ vì nhãn gốc có tên DDoS.

Nếu thầy yêu cầu tấn công thực hiện qua bản tin MQTT, đó là phạm vi khác: cần dữ liệu có phiên MQTT thật, nhãn kịch bản, bằng chứng ảnh hưởng và xem lại đặc trưng/model phù hợp. Không đổi nhãn hai PCAP hiện có thành “MQTT flood”. Chưa cần train lại chỉ để sửa cách gọi tầng giao thức của đề tài hiện tại.

Bảng thử nghiệm nên có ba trạng thái: bình thường; có lưu lượng tấn công khi chỉ quan sát; có cùng lưu lượng khi bật bảo vệ. Thực hiện trong lab của nhóm và ghi rõ cấu hình/tải thực đạt được. Đo:

• Client mới: tỷ lệ kết nối thành công và thời gian kết nối.

• Client đã kết nối sẵn: tỷ lệ gửi/nhận message, độ trễ và mất/ngắt kết nối.

• Broker: tiến trình còn chạy hay không, CPU/RAM, log lỗi và số kết nối.

• Gateway: số gói nhận/chuyển tiếp/bỏ, cảnh báo AI và quyết định chặn.

sensor_client.py hiện tạo một lần connect/publish/disconnect cho mỗi reading. elapsed_ms bao gồm cả lượt đó, không phải độ trễ riêng một bản tin. Cần đo thêm client giữ kết nối lâu nếu muốn đại diện thói quen MQTT thực tế. Việc đo có thể cho thấy không ảnh hưởng ở mức tải đã thử; cần báo đúng kết quả đó, không ép một kết luận “sập”.

Cách trả lời thầy có thể dùng: “Dữ liệu attack hiện tại của nhóm là TCP SYN flood và TCP SYN port scan. MQTT là dịch vụ phía sau gateway cần được bảo vệ, không phải giao thức mà các gói attack này đang trao đổi. Nhóm sẽ tách bằng chứng phát hiện lưu lượng khỏi bằng chứng gián đoạn dịch vụ, đo cả client MQTT đang kết nối và client mở kết nối mới. Nếu yêu cầu chuyển sang tấn công tầng MQTT, nhóm cần bổ sung dữ liệu và cách đánh giá phù hợp.”

8. Nguồn và tệp đối chiếu

[1] Trang chính thức IoT-23: https://www.stratosphereips.org/datasets-iot23

[2] Zenodo, IoT-23 v1.0.0, DOI 10.5281/zenodo.4743746: https://zenodo.org/records/4743746

[3] Chỉ mục đầy đủ 23 kịch bản:
https://mcfp.felk.cvut.cz/publicDatasets/IoT-23-Dataset/IndividualScenarios/

[4] README capture 34: https://mcfp.felk.cvut.cz/publicDatasets/IoT-23-Dataset/IndividualScenarios/CTU-IoT-Malware-Capture-34-1/README.md

[5] Nguồn scan Chris Sanders, đúng bản đã dùng:
https://github.com/chrissanders/packets/tree/0412fb1dc0aa671128f9da6689e0dbd4459c0b51

[6] Mosquitto ChangeLog chính thức, bản ghim v2.0.22; xem mục 2.0.10 và CVE-2021-28166:
https://raw.githubusercontent.com/eclipse-mosquitto/mosquitto/v2.0.22/ChangeLog.txt

Các tệp JSON đi kèm lưu checksum và phạm vi đọc. DANH_MUC_23_KICH_BAN.csv liệt kê đủ 23 thư mục; ALL_20_MALWARE_LABEL_SUMMARIES.json giữ nhãn công bố của 20 kịch bản độc hại. PCAP_PROTOCOL_EVIDENCE.json và FULL_CAPTURE34_PROTOCOL_SUMMARY.json giữ các số đếm trực tiếp. Không kèm hay chạy malware.